

Contact Finder — Build Guide

Oct 1, 2026 · @Grant

Implementation handoff for the pipeline specified in `Doc` . Read the spec for what and why; this guide is how, file by file.

How to use this guide

Paste the kickoff prompt at the end of this doc into Claude Code from an empty folder, with this guide and the spec exported as Markdown into `/docs` . Work one milestone at a time; each milestone's checklist is the definition of done.

- **Spec** = scope, pipeline stages, data model, architecture, decision layer, orchestration, compliance. Change the spec when the product changes.
- **This guide** = repo layout, env, code skeletons, prompt files, fixtures, tests, milestone checklists. Change this when the implementation changes.
- Code in this guide is a skeleton, not a finished implementation: signatures, shapes, and the non-obvious parts are given; the coding agent fills in the rest.
- Stack: TypeScript everywhere (core, edge, web). pnpm workspace. Vitest for tests. Vite + React for the web app. Supabase Edge Functions (Deno) for the proxy.

Assumptions made here: Supabase for the edge function, HubSpot as the first CRM target, MX-only verification in v1. Each is listed in the spec's Open decisions table; change them there first if you disagree. should be agnositc crm options. pretty basic databse that we can use doesnt need to be crazy. simple to start.

Prerequisites and env

Accounts: Anthropic API key with web search enabled on the org; Supabase project (free tier); a Git host. Nothing else for v1.

Local tools: Node 20+, pnpm 9+, Supabase CLI, Deno (installed by the Supabase CLI).

Variable	Where	Purpose
<code>ANTHROPIC_API_KEY</code>	Supabase secret only	Never shipped to the browser

Variable	Where	Purpose
<code>CF_MODEL_EXTRACT</code>	edge	Model id for extraction and pattern stages (Sonnet-class)
<code>CF_MODEL_CLASSIFY</code>	edge	Model id for the classifier (Haiku-class)
<code>CF_MAX_SEARCHES_PER_COMPANY</code>	edge	Default <code>4</code>
<code>CF_RATE_LIMIT_PER_MIN</code>	edge	Default <code>30</code> calls per session
<code>CF_CACHE_TTL_DAYS</code>	edge	Default <code>30</code> for <code>domain</code> → <code>patterns</code>
<code>VITE_EDGE_URL</code>	web <code>.env</code>	URL of the deployed <code>pipeline</code> function

Set secrets with `supabase secrets set ANTHROPIC_API_KEY=...`. Keep a `.env.example` in the repo with every variable and a comment; never commit `.env`. Model ids are config, not code, so a model swap is a redeploy with no PR.

Repo scaffold

One pnpm workspace, three packages plus docs and prompts. `core` has zero runtime dependencies on React, Deno, or Supabase so it can be imported by the edge function, the web app, and later the MCP server unchanged.

```

contact-finder/
├── docs/
│   ├── spec.md           # exported from the spec doc
│   └── build-guide.md    # exported from this doc
├── prompts/              # markdown, read at runtime by the edge fn
│   ├── classify_extract.md
│   ├── resolve_domain.md
│   ├── discover_pattern.md
│   ├── find_people.md
│   └── rescue_agent.md
├── fixtures/             # sample pastes + expected outputs
│   ├── linkedin_results.txt
│   ├── team_page.txt
│   ├── mixed_notes.txt
│   ├── companies_only.txt
│   └── urls.txt

```

```

├── └─ expected/*.json
├── packages/
│   ├── core/                                # pure TS, no I/O
│   │   ├── src/types.ts
│   │   ├── src/normalize.ts
│   │   ├── src/candidates.ts
│   │   ├── src/stages/                    # one file per stage, each (input, ctx) =>
│   │   └─ StageResult
│   │       ├── src/runner.ts              # runPipeline + runRescue
│   │       ├── src/decisions/             # DecisionProvider + ClaudeDecisions
│   │       ├── src/verify/               # Verifier + MxVerifier
│   │       ├── src/export/csv.ts
│   │       └─ test/
│   ├── edge/                              # supabase/functions/pipeline/index.ts
│   └─ web/                                # Vite + React
│       └─ src/{App,Input,Preview,ResultsTable,ExportBar}.tsx
├── .env.example
├── pnpm-workspace.yaml
└── package.json

```

Rule for the coding agent: every stage, the normalizer, and the CSV exporter get a Vitest file before the web app is touched. Milestone 1 is green tests with no network.

Core: types, normalizer, candidates

The types are the spec's data model verbatim, plus `StageResult`, the envelope every stage returns.

```

// packages/core/src/types.ts
export type Template =
  | `${first}.${last}` | `${first}${last}` | `${first}_${last}` | `${first}-${last}`
  | `${f}${last}` | `${f}.${last}` | `${first}` | `${last}` | `${last}.${first}` | "
  {first}{1}";

export const DEFAULT_PATTERNS: Pattern[] = [
  { template: `${first}.${last}`, confidence: 0.45 },
  { template: `${first}`, confidence: 0.15 },
  { template: `${f}${last}`, confidence: 0.15 },
];

export type StageResult<T> = {
  ok: boolean;

```

```

data?: T;
confidence?: number;          // 0-1, as reported by the stage
sources: string[];            // URLs the result was read from
tokens_used: number;
searches_used: number;
error?: string;
};

export type Ctx = {
  llm: (req: LlmRequest) => Promise<LlmResponse>; // injected: edge fn
  implements it
  cache: { get(k: string): Promise<any>; set(k: string, v: any, ttlDays:
number): Promise<void> };
  decisions: DecisionProvider;
  budget: { maxSearchesPerCompany: number; maxRescueCalls: number };
};

```

Normalizer and generator are pure and fully unit-tested. Nickname expansion is behind a flag and only fills an empty third slot.

```

// packages/core/src/normalize.ts
export function normalizeName(raw: string): { first: string; last: string;
firstAlt?: string; lastAlt?: string } {
  let s = raw.normalize("NFD").replace(/[\u0300-\u036f]/g, ""); // José →
Jose
  s = s.replace(/(\b(dr|mr|mrs|ms|prof)\.?\s+/gi, ""); //
honorifics
  s = s.replace(/,.*$/, ""); // ",
MBA" / ", PhD"
  s = s.replace(/(\b(jr|sr|ii|iii|iv)\.?\$/i, "").trim(); //
suffixes
  s = s.replace(/['.]/g, "").toLowerCase(); //
O'Brien → obrien
  const parts = s.split(/\s+/).filter(Boolean);
  const first = parts[0] ?? "";
  const lastRaw = parts.at(-1) ?? "";
  const out = { first, last: lastRaw.replace(/-/g, "") }; //
alvarez-ruiiz → alvarezruiiz
  if (lastRaw.includes("-")) (out as any).lastAlt = lastRaw.split("-")[0];
  // also try "alvarez"
  return out;
}

```

```
// packages/core/src/candidates.ts
export function generateCandidates(name: ReturnType<typeof normalizeName>,
domain: string,
                                patterns: Pattern[], opts = { nicknames:
false }): Candidate[] {
  const fill = (t: string, f: string, l: string) =>
    t.replace("{first}", f).replace("{last}", l).replace("{f}", f[0] ??
    "").replace("{l}", l[0] ?? "");
  const seen = new Set<string>();
  const out: Candidate[] = [];
  const push = (p: Pattern, f: string, l: string) => {
    const email = `${fill(p.template, f, l)}@${domain}`;
    if (!seen.has(email) && out.length < 3) { seen.add(email); out.push({
email, pattern: p.template, rank: (out.length + 1) as 1|2|3, verify_status:
"unverified" }); }
  };
  const ranked = (patterns.length ? patterns :
DEFAULT_PATTERNS).slice().sort((a, b) => b.confidence - a.confidence);
  for (const p of ranked) push(p, name.first, name.last);
  if (name.lastAlt) for (const p of ranked) push(p, name.first,
name.lastAlt);
  if (opts.nicknames && out.length < 3) { const alt = NICKNAMES[name.first];
if (alt) push(ranked[0], alt, name.last); }
  return out;
}
```

Required test cases for `normalizeName` → (first, last, lastAlt?): José Álvarez-Ruiz → jose, alvarezruiz, alvarez · Mary Anne O'Brien → mary, obrien · Dr. Li Wei PhD → li, wei · Bob Smith Jr. → bob, smith · Jean-Luc Picard → jeanluc, picard.

Core: runner, decisions, rescue

The runner is a plain loop: companies in parallel (limit 5), then contacts within each company. It calls `ctx.decisions` only at the gate; the gate decides whether `runRescue` is invoked for a row.

```
// packages/core/src/runner.ts
export async function runPipeline(input: string, ctx: Ctx, onRow: (c:
Contact) => void): Promise<RunResult> {
  const ex = await stages.classifyExtract(input, ctx); // stage
1
  const companies = new Map(ex.data!.companies.map(c => [c.id, c]));
```

```

    await pMap([...companies.values()], 5, async company => {
      const cached = await ctx.cache.get(`company:${company.name}`);
      if (cached) Object.assign(company, cached);
      else {
        const d = await stages.resolveDomain(company, ctx);          // stage
2        if (d.ok) { company.domain = d.data!.domain; company.domain_confidence
= d.confidence; company.domain_source_url = d.sources[0]; }
        if (company.domain) {
          const p = await stages.discoverPattern(company.domain, ctx); // stage
3          company.patterns = p.ok ? p.data!.patterns : [];
          company.mx_ok = await ctx.verify.mx(company.domain);      //
stage 5 (v1)
        }
        await ctx.cache.set(`company:${company.name}`, company,
ctx.budget.cacheTtlDays);
        }
        for (const contact of ex.data!.people.filter(p => p.company_id ===
company.id)) {
          contact.candidates = company.domain ?
generateCandidates(normalizeName(`${contact.first} ${contact.last}`),
company.domain, company.patterns) : [];
          contact.status = !company.domain ? "no_domain" :
company.patterns.length ? "ok" : "no_pattern";
          if (await shouldRescue(contact, company, ctx)) {
            const fixed = await runRescue(contact, company, ctx);    // see
below
            if (fixed.ok) Object.assign(contact, fixed.data);
          }
          onRow(contact);
        }
      });
    }
    return summarize(ex, companies);
  }

// gate: v1 rule, v2 calibrated
async function shouldRescue(c: Contact, co: Company, ctx: Ctx):
Promise<boolean> {
  if (c.status !== "ok") return true;
  if (!ctx.decisions.calibrated) return false;                      //
ClaudeDecisions: status-only gate
  const p = await ctx.decisions.score(JSON.stringify({ company: co, contact:

```

```

c }), "Is this domain and pattern correct given the evidence?");
    return p < ctx.budget.rescueThreshold; //
default 0.6
}

```

```

// packages/core/src/decisions/index.ts
export interface DecisionProvider {
    name: string;
    calibrated: boolean; // true only for a model that returns calibrated
    probabilities (Jev)
    classify(labels: string[], context: string): Promise<Record<string,
    number>>;
    choose(options: string[], context: string): Promise<{ index: number; probs:
    number[] }>;
    score(context: string, question: string): Promise<number>;
}
// ClaudeDecisions: one tool-use call with an enum schema; calibrated =
false.
// JevDecisions (v2): one HTTP call per method; calibrated = true. Same file
shape, swap by config.

```

The rescue agent is a bounded tool-use loop. The tools are the stage functions themselves plus the server-side web tools; the loop ends on a `finish` tool call or when the budget is spent.

```
// packages/core/src/runner.ts (continued)
export async function runRescue(c: Contact, co: Company, ctx: Ctx):
Promise<StageResult<Partial<Contact>>> {
  const tools = [findDomainTool, findPatternTool, findPeopleTool,
finishTool]; // + web_search / web_fetch enabled server-side
  const messages = [{ role: "user", content: prompts.rescueAgent({ contact:
c, company: co, tried: co.patterns, status: c.status }) }]];
  for (let i = 0; i < ctx.budget.maxRescueCalls; i++) { //
default 8
    const res = await ctx.llm({ messages, tools, maxSearches: 2 });
    const call = res.toolCalls[0];
    if (!call || call.name === "finish") return parseFinish(call); // {ok,
data:{domain?, patterns?, candidates?}, sources}
    const out = await runTool(call, ctx); //
dispatches to the stage fns
    messages.push(res.assistantTurn, toolResult(call, out));
    if (ctx.decisions.calibrated && await
ctx.decisions.score(JSON.stringify(out), "Is this row now good enough to
stop?") > 0.8) break;
  }
  return { ok: false, sources: [], tokens_used: 0, searches_used: 0, error:
"rescue budget exhausted" };
}
```

Every stage function has the same signature, `(input, ctx) =>`

`Promise<StageResult<T>>`, and does exactly one LLM call (stages 2–3 with web search enabled, stage 1 without). Stage 4 and the CSV export are pure code with no `ctx`.

Edge function

One Supabase Edge Function, `pipeline`, with two routes. It is the only place the API key exists and the only server code in v1. The browser never calls Anthropic directly.

Route	Body	Does
POST /llm	<code>{ stage, input, options }</code>	Loads <code>prompts/<stage>.md</code> , builds the Messages request with the stage's tool schema and <code>tool_choice</code> , enables <code>web_search</code> / <code>web_fetch</code> for stages that need them, forwards, returns <code>{content, toolCalls, usage}</code>
POST /mx	<code>{ domain }</code>	DNS MX lookup, returns <code>{ ok }</code>

```
// packages/edge/supabase/functions/pipeline/index.ts (Deno)
import Anthropic from "npm:@anthropic-ai/sdk";
const client = new Anthropic({ apiKey: Deno.env.get("ANTHROPIC_API_KEY") });
const limiter = new Map<string, number[]>(); //
sessionId → timestamps

Deno.serve(async req => {
  const session = req.headers.get("x-session") ?? "anon";
  if (!allow(session, Number(Deno.env.get("CF_RATE_LIMIT_PER_MIN") ?? 30)))
    return json({ error: "rate_limited" }, 429);
  const url = new URL(req.url);
  if (url.pathname.endsWith("/mx")) return json({ ok: (await
Deno.resolveDns((await req.json()).domain, "MX").catch(() => [])).length > 0
});

  const { stage, input, options } = await req.json();
  const prompt = await Deno.readFile(`./prompts/${stage}.md`);
  const spec = STAGES[stage]; // { model,
tool, webTools, maxSearches }
  const res = await client.messages.create({
    model: Deno.env.get(spec.modelEnv)!,
    max_tokens: 2048,
    system: prompt,
    messages: [{ role: "user", content: typeof input === "string" ? input :
JSON.stringify(input) }],
    tools: [spec.tool, ...(spec.webTools ? webTools(spec.maxSearches) : [])],
    tool_choice: options?.agentic ? { type: "auto" } : { type: "tool", name:
spec.tool.name },
  });
  return json({
    toolCalls: res.content.filter(b => b.type === "tool_use").map(b => ({
```

```
name: b.name, input: b.input })),
  sources: res.content.flatMap(b => (b as any).citations?.map((c: any) =>
c.url) ?? []),
  usage: res.usage,
});
});
```

Notes for the implementer:

- `webTools(n)` returns the server-side web search (and fetch) tool definitions with `max_uses: n`; check the current tool type strings in the Anthropic docs rather than hard-coding from memory, and keep them in one constant.
- `STAGES` maps stage name → `{ modelEnv, tool, webTools, maxSearches }`; the tool schemas are imported from `core` so the edge function and the browser agree on shapes.
- Cache: a `cf_cache` table (key text primary key, value jsonb, expires_at timestampz) read/written by the function; the web app also keeps its own localStorage cache so a cold edge cache still feels fast.
- Log one row per call to a `cf_usage` table: session, stage, input tokens, output tokens, searches. The UI's cost counter reads the response `usage`; the table is for your own bill tracking.
- CORS: allow the web app's origin only; `x-session` is a random UUID the browser generates once and stores.

Prompt files

Each prompt is a Markdown file in `/prompts` , used verbatim as the system prompt. The tool schema for each stage lives in `core/src/stages/<stage>.ts` so the shapes are typed once. Prompt text is in the spec; what is new here is the file list, the schema per file, and the rescue prompt.

File	Tool name	Web tools	Returns
<code>classify_extract.md</code>	<code>extract_contacts</code>	none	<code>{mode, companies[], people[], urls[], notes}</code>

File	Tool name	Web tools	Returns
<code>resolve_domain.md</code>	<code>report_domain</code>	search, max 2	<code>{domain, confidence, source_url, alternatives[]}</code>
<code>discover_pattern.md</code>	<code>report_patterns</code>	search, max 3	<code>{patterns: [{template, confidence, source_url?, evidence?}]}</code>
<code>find_people.md</code>	<code>extract_contacts</code>	search + fetch, max 3	same as stage 1, filtered by role
<code>rescue_agent.md</code>	<code>find_domain,</code> <code>find_email_pattern,</code> <code>find_people, finish</code>	search + fetch, max 2 per turn	agentic; ends with <code>finish</code>

`rescue_agent.md`:

You repair one contact row that the fixed pipeline could not complete. You are given the contact, the company, what was already tried, and the failure status. Use the tools to try alternatives:

- a parent company, a rebrand, or a regional domain (e.g. `acme.co.uk`)
- the company's own team or about page, fetched, not just snippets
- a different search phrasing for the email format

Stop as soon as you have a domain with a source URL and at least one pattern with a source URL,

or when further attempts are unlikely to help. Then call `'finish'` with either the repaired fields

and their source URLs, or `'gave_up: true'` and a one-line reason. Never invent a domain or a pattern;

every value you return must come from a tool result.

```
finish schema: { domain?: string, domain_source_url?: string, patterns?:  
Pattern[], gave_up?: boolean, reason?: string }.
```

Template tokens allowed in any pattern output: `{first}` `{last}` `{f}` `{l}` with separators `.` `_` `-` or none. The stage code validates the template against the `Template` union and drops anything else, so a creative model answer cannot leak into the export.

Web app

Five components, one reducer, no router. The browser builds a `Ctx` whose `llm` function calls the edge route, then runs `core`'s `runPipeline` directly, so the pipeline logic is identical in tests and in the UI.

Component	Owns	Notes
<code>App</code>	reducer state, <code>Ctx</code> , session id	Persists state to <code>localStorage</code> on every reducer change, keyed by session
<code>Input</code>	textarea, role filter, Parse / Run buttons	Parse calls stage 1 only; Run calls <code>runPipeline</code>
<code>Preview</code>	editable mini-table of extracted people and companies	Edits write back to state before Run; shows "Found N people at M companies"
<code>ResultsTable</code>	one row per contact, filters, Retry	Columns per the spec's UI section; <code>Retry</code> re-runs stages 2–4 for that row's company, then rescue
<code>ExportBar</code>	Copy TSV, Download CSV, cost counter	CRM button rendered disabled with a tooltip in v1

```
// packages/web/src/state.ts
type State = {
  session: string;
  input: string; roleFilter: string;
  extracted?: ExtractResult;           // after Parse
  companies: Record<string, Company>;
  contacts: Record<string, Contact>;
  running: boolean;
  usage: { tokens: number; searches: number };
  error?: string;
};

type Action =
  | { type: "input"; input: string } | { type: "parsed"; extracted:
ExtractResult }
  | { type: "row"; contact: Contact } | { type: "company"; company: Company }
  | { type: "usage"; tokens: number; searches: number }
  | { type: "run_start" } | { type: "run_end"; error?: string } | { type:
"clear" };

```

Run loop in `App: dispatch(run_start) → runPipeline(input, ctx, c => dispatch({type:"row", contact:c})) → dispatch(run_end)`. Each `ctx.11m` response also dispatches `usage`. A 429 from the edge function is caught inside `ctx.11m`, which waits 10 s and retries once before surfacing an error; the UI shows "rate limited, resuming" while it waits.

Styling: Tailwind, no component library. Cost estimate shown as `≈ $` from a constant per-token and per-search price in `config.ts`, labelled as an estimate.

Fixtures and tests

Five pasted-text fixtures drive both the unit tests and the manual QA. Each has an `expected/<name>.json` with the people and company counts and, where known, the real domain. Write them with real public companies so stage 2–3 tests can run live.

Fixture	What it contains	Expected
<code>linkedin_results.txt</code>	10 search-result cards copied from LinkedIn: name, badge, title, company, location, "Connect"	10 people, 4–6 companies, zero badges or locations in output

Fixture	What it contains	Expected
<code>team_page.txt</code>	A company /team page copied whole: headings, names, titles, bios	1 company, N people, titles attached to the right names
<code>mixed_notes.txt</code>	Freeform notes: "Acme (acme.com) — Jane Doe VP Sales; Beta Corp, need CFO"	2 companies, 1 person, website captured for Acme
<code>companies_only.txt</code>	8 company names, one per line	mode <code>companies</code> , 0 people
<code>urls.txt</code>	3 team-page URLs	mode <code>urls</code> , 3 urls

Test layers:

- Pure unit tests (no network, run on every commit):** `normalizeName`, `generateCandidates`, CSV export, template validation, the status-only rescue gate, the runner with a mocked `ctx.llm` that returns canned tool calls.
- Contract tests (mocked LLM):** each stage parses a recorded real API response into the right `StageResult`; record one response per stage into `fixtures/recorded/` with a script so tests don't hit the API.
- Live smoke test (manual, `pnpm smoke`):** runs `linkedin_results.txt` end to end against the deployed edge function and prints the table plus total searches and tokens. Pass = $\geq 80\%$ rows `ok`, ≤ 4 searches per company.

```
// packages/core/test/runner.test.ts (shape)
it("routes a no_pattern row to rescue and accepts a repaired row", async ()
=> {
  const ctx = mockCtx({ discoverPattern: { ok: true, data: { patterns: [] }
},
                        rescue: finish({ patterns: [{ template: "{first}.
{last}", confidence: 0.7, source_url: "https://x" }] }) });
  const rows: Contact[] = [];
  await runPipeline(fixture("mixed_notes"), ctx, r => rows.push(r));
  expect(rows[0].status).toBe("ok");
  expect(rows[0].candidates[0].email).toBe("jane.doe@acme.com");
});
```

Milestone checklists

Same six milestones as the spec, as tickable tasks. Each milestone ends with a commit and a passing `pnpm test`.

M1 — Core, no network

- ☐ `pnpm` workspace with `core`, `edge`, `web` packages; Vitest wired
- ☐ `types.ts`, `normalize.ts`, `candidates.ts`, `export/csv.ts` with tests, including the five tricky names
- ☐ Five fixtures and their `expected/*.json`
- ☐ `runner.ts` passing with a fully mocked `ctx`

M2 — Edge function + stage 1

- ☐ `pipeline` function deployed; `ANTHROPIC_API_KEY` set as a secret; `.env.example` complete
- ☐ Rate limit returns 429 after the configured calls per minute
- ☐ `classify_extract` live against all five fixtures: counts within ± 1 of expected
- ☐ Recorded responses saved for contract tests

M3 — Stages 2–3 with web search

- ☐ `resolve_domain` and `discover_pattern` live; `cf_cache` table and read-through cache
- ☐ 10-company list: ≥ 8 correct domains, ≥ 6 patterns with a source URL, ≤ 40 searches total
- ☐ `cf_usage` logging

M4 — Web UI

- ☐ Input → Parse → Preview (editable) → Run → live ResultsTable → CSV and TSV export
- ☐ `localStorage` persistence survives refresh; Clear wipes it
- ☐ 30-contact fixture completes in under 2 minutes

M5 — Verify, rescue, polish

- ☐ `MxVerifier` via `/mx`; dead-domain fixture marks rows `no_domain`
- ☐ `runRescue` with the 8-call budget; Retry button per row
- ☐ Filters, cost counter within 10 % of `cf_usage`

M6 — Company-first

- ☐ `find_people` with role filter; 3 companies + "VP Sales" returns ≥ 1 match each where a public team page exists
- ☐ README with deploy steps and compliance notes

v2 backlog (not now): `JevDecisions` + calibrated rescue gate · ICP fit scoring column · HubSpot adapter · remote MCP server exposing the stage tools · Agent SDK researcher subagents for the MCP server.

Kickoff prompt for Claude Code

Export both docs to Markdown into `/docs`, open Claude Code in the empty repo folder, and paste this:

You are building Contact Finder, a contact-finding and email-pattern pipeline. Read `docs/spec.md` and `docs/build-guide.md` fully before writing any code. The guide's repo scaffold, types, and code skeletons are the source of truth; where the two docs disagree, the guide wins for implementation detail and the spec wins for scope.

Work milestone by milestone from the guide's checklists. For each milestone:

1. Restate the checklist and the acceptance test in your own words.
2. Implement, with Vitest tests written before or alongside the code.
3. Run ``pnpm test``; do not move on with red tests.
4. Commit with the milestone name, then stop and summarize what to verify manually.

Constraints that override anything else:

- ``packages/core`` imports nothing from React, Deno, or Supabase.
- The Anthropic API key exists only in the edge function.
- Never add code that fetches `linkedin.com` or any login-walled page.
- Prompts are Markdown files in `/prompts`, read at runtime, never inlined in code.
- Model ids, budgets, and thresholds come from `env` or `config.ts`, never literals in logic.
- Look up the current Anthropic web search / web fetch tool definitions in the official docs before writing the edge function; do not rely on memory for tool type strings.

Start with Milestone 1. Before writing files, show me the planned file list for M1 and wait for my go-ahead.